

```
In [1]: import numpy as np
arr = np.array([1,2,3,4,5])
print(arr)
```

[1 2 3 4 5]

```
In [2]: a = [1,2,3]
b = [4,5,6]
c = a+b
print(c)
```

[1, 2, 3, 4, 5, 6]

```
In [3]: a = [1,2,3]
b = [4,5,6]
res = []
for i in range(len(a)):
    res.append(a[i]+b[i])
print(res)
```

[5, 7, 9]

```
In [5]: import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])
print(a+b)
print(a-b)
print(a*b)
print(a//b)
print(a%b)
print(a**b)
```

[5 7 9]

[-3 -3 -3]

[4 10 18]

[0 0 0]

[1 2 3]

[1 32 729]

```
In [12]: import numpy as np
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[[1,2,3],[4,5,6],[7,8,9]]])
print(arr1)
print(arr2)
print(arr1.shape)
print(arr2.shape)
print(arr1.ndim)
print(arr2.ndim)
print(arr1.size)
print(arr2.size)
print(arr1.dtype)
```

```
[[1 2 3]
 [4 5 6]]
[[[1 2 3]
  [4 5 6]
  [7 8 9]]]
(2, 3)
(1, 3, 3)
2
3
6
9
int64
```

Array creation in numpy

```
In [29]: import numpy as np
a = np.array([1,2,3,4,5])
b = np.zeros(5)
print(a)
print(b)
c = np.zeros((2,2))
print(c)
d = np.ones((2,2))
print(d)
f = np.arange(1,10,2)
print(f)
print(np.linspace(0,20,5))
print(np.random.rand(5))
print(np.random.randint(1,100,20))
print(np.random.randn(5))
```

```
[1 2 3 4 5]
[0. 0. 0. 0. 0.]
[[0. 0.]
 [0. 0.]]
[[1. 1.]
 [1. 1.]]
[1 3 5 7 9]
[ 0.  5. 10. 15. 20.]
[0.76410899 0.13442414 0.67650797 0.35473606 0.44735603]
[19 46 70 21 69 59  6 55 66 16 89 19 59 13 86 55 11 89 16 31]
[ 0.83091466  0.30804854 -0.23028133  0.33484368 -0.03620646]
```

```
In [30]: # Numpy Indexing and Slicing
```

```
In [54]: import numpy as np
a = np.array([5,10,15,20,25])
print(a[2])
b = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(b[1][1])
print(a[:4])
print(a[::2])
print(b[2])
print(b[:,1])
print(b[1,:])
print(b[-2])
print(a[-4])
print(a[:-2])
print(a[-2::])
print(a[a<20])
```

```

print(a[a>=15])
print(a[[0,4,3]])
print(a[a%2==0])
a[2] = 30
print(a)
b[2] = 11
print(b)
b[2][2] =12
print(b)

```

```

15
5
[ 5 10 15 20]
[ 5 15 25]
[7 8 9]
[2 5 8]
[4 5 6]
[4 5 6]
10
[25 15  5]
[20 25]
[ 5 10 15]
[15 20 25]
[ 5 25 20]
[10 20]
[ 5 10 30 20 25]
[[ 1  2  3]
 [ 4  5  6]
 [11 11 11]]
[[ 1  2  3]
 [ 4  5  6]
 [11 11 12]]

```

Mathematical Operation in numpy

```

In [56]: #scalar Broadcasting
import numpy as np
a = np. array([1,2,3])
print(a+10)
#2d broadcasting
b = np.array([[1,2,3],[2,4,6]])
print(b+10)
#row broadcasting
print(b+a)

```

```

[11 12 13]
[[11 12 13]
 [12 14 16]]
[[2 4 6]
 [3 6 9]]

```

```

In [62]: #Matrix Operation
import numpy as np
a = np.array([[1,2],[3,4]])
b = np.array([[4,3],[2,1]])
print(a*b)
print(np.dot(a,b))
print(a@b)
print(np.matmul(a,b))

```

```
#transpose  
print(a.T)
```

```
[[4 6]  
 [6 4]]  
[[ 8  5]  
 [20 13]]  
[[ 8  5]  
 [20 13]]  
[[ 8  5]  
 [20 13]]  
[[4 6]  
 [6 4]]
```

```
In [2]: #round function  
import numpy as np  
arr = np.array([1.2333,5.666])  
print(np.round(arr,2))  
#floor  
print(np.floor(arr))  
#ceil  
print(np.ceil(arr))
```

```
[1.23 5.67]  
[1.  5.]  
[2.  6.]
```

Statistical Function

```
In [16]: import numpy as np  
a = np.array([10,20,30,40,50])  
print(np.mean(a))  
print(np.median(a))  
print(np.std(a))  
print(np.max(a))  
print(np.min(a))  
print(np.percentile(a,50))  
print(np.var(a))  
print(np.argmin(a))  
print(np.argmax(a))  
b = np.array([1,2,3,4,5])  
c = np.array([10,8,6,4,2])  
print(np.corrcoef(b,c))  
#aggregate Function  
d = np.array([[1,2,3],[4,5,6]])  
print(np.sum(d))  
print(np.sum(d,axis=0))  
print(np.sum(d,axis=1))
```

```

30.0
30.0
14.142135623730951
50
10
30.0
200.0
0
4
[[ 1. -1.]
 [-1.  1.]]
21
[5 7 9]
[ 6 15]

```

```

In [33]: import numpy as np
StudentScore = np.random.randint(1,101,size = (10,5))
print("Student Score:",StudentScore)
print("Average Score:",np.mean(np.sum(StudentScore,axis=1)))
print("Highest Score:",np.max(np.sum(StudentScore,axis=1)))
print("Lowest Score:",np.min(np.sum(StudentScore,axis=1)))
normalized = (StudentScore - StudentScore.min()) / (StudentScore.max() -
print(normalized)

```

```

Student Score: [[77 62 11 40 66]
 [73 21 19 23 54]
 [64 85 40 52 20]
 [73 25 68 36 52]
 [66 77 98  4 47]
 [92 60 36 86 88]
 [46 88 32 98  1]
 [ 2 84 46  5 73]
 [ 3 48 34 48 85]
 [13 64 23  7 59]]
Average Score: 247.4
Highest Score: 362
Lowest Score: 166
[[0.78350515 0.62886598 0.10309278 0.40206186 0.67010309]
 [0.74226804 0.20618557 0.18556701 0.22680412 0.54639175]
 [0.64948454 0.86597938 0.40206186 0.5257732  0.19587629]
 [0.74226804 0.24742268 0.69072165 0.36082474 0.5257732 ]
 [0.67010309 0.78350515 1.          0.03092784 0.4742268 ]
 [0.93814433 0.60824742 0.36082474 0.87628866 0.89690722]
 [0.46391753 0.89690722 0.31958763 1.          0.          ]
 [0.01030928 0.8556701  0.46391753 0.04123711 0.74226804]
 [0.02061856 0.48453608 0.34020619 0.48453608 0.86597938]
 [0.12371134 0.64948454 0.22680412 0.06185567 0.59793814]]

```

```

In [32]: import numpy as np
StudentScore = np.random.randint(1,101,size = (10,5))
print("Student Score:",StudentScore)
print("Average Score:",np.mean(np.sum(StudentScore,axis=0)))
print("Highest Score:",np.max(np.sum(StudentScore,axis=0)))
print("Lowest Score:",np.min(np.sum(StudentScore,axis=0)))

```

```
Student Score: [[ 4 59 77 41 35]
 [69 92  9 42 70]
 [ 8 40 81 45 84]
 [88 28 58 31 76]
 [70 12 29 28 29]
 [77 45 34 44 25]
 [70  6  3 81 94]
 [36 60 59 27 53]
 [84 13 13 84 29]
 [36  2 79 93 50]]
Average Score: 480.4
Highest Score: 545
Lowest Score: 357
```

```
In [35]: import numpy as np
arr = np.array([10,20,30,40,50])
print(arr)
print(np.sum(arr[arr>25]))
```

```
[10 20 30 40 50]
120
```

PANDAS

```
In [9]: #series
import pandas as pd
a = pd.Series([1,2,3,4])
print(a)
print(a.values)
print(a.index)
print(a.dtype)
#accessing elements
print(a[1])
#accessing element using loc
print(a.loc[2])
print(a.iloc[2])
#slicing
print(a[1:3])
print(a.loc[1:3])
print(a.iloc[2:5])
```

```

0    1
1    2
2    3
3    4
dtype: int64
[1 2 3 4]
RangeIndex(start=0, stop=4, step=1)
int64
2
3
3
1    2
2    3
dtype: int64
1    2
2    3
3    4
dtype: int64
2    3
3    4
dtype: int64

```

Create Index in Series

```

In [31]: import pandas as pd
arr = pd.Series([1,2,3,4,5], index = ['A','B','C', 'D', 'E'])
print(arr)
print(arr[['A','C','E']])
print(arr['E'])
#vectorized operation - no loop required
print(arr+5)
print(arr['A']+5)
print(arr)
print(arr.iloc[1]) # based on position
print(arr.loc['A']) # based on index value
print(arr.sum())
print(arr.mean())
print(arr.max())
print(arr.min())
print(arr.std())
print(arr.median())
print(arr[arr%2==0])
#count values
print(len(arr))
print(arr.value_counts())

```

```

A    1
B    2
C    3
D    4
E    5
dtype: int64
A    1
C    3
E    5
dtype: int64
5
A    6
B    7
C    8
D    9
E   10
dtype: int64
6
A    1
B    2
C    3
D    4
E    5
dtype: int64
2
1
15
3.0
5
1
1.5811388300841898
3.0
B    2
D    4
dtype: int64
5
1    1
2    1
3    1
4    1
5    1
Name: count, dtype: int64

```

String Operation

```

In [23]: import pandas as pd
name = pd.Series(['midhun','vishnu','shankar','sakthi'])
print(name)
print(name.str.upper())
print(name.str.len())
print(name.str.capitalize())

```



```

0    midhun
1    vishnu
2    shankar
3    sakthi
dtype: object
0    MIDHUN
1    VISHNU
2    SHANKAR
3    SAKTHI
dtype: object
0    6
1    6
2    7
3    6
dtype: int64
0    Midhun
1    Vishnu
2    Shankar
3    Sakthi
dtype: object

```

Dictionaries in pandas

```

In [27]: import pandas as pd
a = pd.Series({"Midhun":16,"Ram":30,"Akash":1})
print(a)
print(a["Midhun"])

```

```

Midhun    16
Ram       30
Akash     1
dtype: int64
16

```

```

In [30]: import pandas as pd
s1 = pd.Series([10,20,30],index=['A','B','C'])
s2 = pd.Series([1,2,3],index=['B','C','D'])
print(s1+s2)
s = pd.Series([1,2,2,3,4,5,5])
print(s.count())
print(s.unique())
print(s.nunique())

```

```

A    NaN
B    21.0
C    32.0
D    NaN
dtype: float64
7
[1 2 3 4 5]
5

```

Dataframe in Pandas

```

In [29]: import pandas as pd
#method1 : from dictionary
data = { 'name':['Midhun','Nagul','Ram'],'rollno':[16,20,30], 'Age':[24,24,24]}
df=pd.DataFrame(data)
print(df)
#method2 : from series

```

```

s1 = pd.Series([1,2,3,4,5])
s2 = pd.Series([5.6,9.54,9.4,54.6,6])
s3 = pd.Series([True,False,True,False])
df = pd.DataFrame({'sno':s1,'cgpa':s2, 'attendance':s3})
print(df)
#list of list
data = [['Midhun',24,98],['Ram',23,89],['Akash',24,97]]
df=pd.DataFrame(data,columns=["Name","Age","Marks"])
print(df)
#list of dictionary to Data Frame
data=[{"name":"Midhun","course":"Data Science","Grade":"A"},
      {"name":"Madhu","course":"AIML","Grade":"A+"},
      {"name":"Akash","course":"CS","Grade":"A++"}]
df = pd.DataFrame(data, index=["Std1","Std2","Std3"])
print(df)
#adding one extra Columns
df['mark'] = [98,87,67]
print(df)
# adding one row
Std4 = pd.DataFrame([{"name":"Mahi","course":"Data Science","Grade":"0",
df = pd.concat([df,Std4])
print(df)
#create multiple rows
students = pd.DataFrame([{"name":"Ragu","course":"CSDA","Grade":"B","mark"
                        {"name":"Raj","course":"AIML","Grade":"D","mark"
df = pd.concat([df,students])
print(df)
#column Access by bracket notation and dot notation
print(df["name"])
print(df.mark)
print(df[["course","Grade"]])
#shape attribute
print(df.shape)
#find number of columns
print(df.shape[1])
#find number of rows
print(df.shape[0])
# find datatype of all columns
print(df.dtypes)
#info
print(df.info())
#describe
print(df.describe())
#select specific row
print(df.loc[['Std1','Std4']])
print(df.iloc[0])
#updatevalue
df.loc['Std1', 'mark'] = 87
print(df)
#delete row
df = df.drop('Std1')
print(df)
#delete column
df=df.drop(["Grade"],axis=1)
print(df)
#sorting
df.sort_values('mark', inplace=True)
print(df)
df.sort_values('mark', ascending=False, inplace=True)
print(df)

```

```
#statistical Function  
print(df['mark'].max())
```

	name	rollno	Age
0	Midhun	16	24
1	Nagul	20	20
2	Ram	30	23

	sno	cgpa	attendance
0	1	5.60	True
1	2	9.54	False
2	3	9.40	True
3	4	54.60	False
4	5	6.00	NaN

	Name	Age	Marks
0	Midhun	24	98
1	Ram	23	89
2	Akash	24	97

	name	course	Grade
Std1	Midhun	Data Science	A
Std2	Madhu	AIML	A+
Std3	Akash	CS	A++

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67
Std4	Mahi	Data Science	0	99

	name	course	Grade	mark
Std1	Midhun	Data Science	A	98
Std2	Madhu	AIML	A+	87
Std3	Akash	CS	A++	67
Std4	Mahi	Data Science	0	99
Std5	Ragu	CSDA	B	50
Std6	Raj	AIML	D	75

Std1	Midhun
Std2	Madhu
Std3	Akash
Std4	Mahi
Std5	Ragu
Std6	Raj

Name: name, dtype: object

Std1	98
Std2	87
Std3	67
Std4	99
Std5	50
Std6	75

Name: mark, dtype: int64

	course	Grade
Std1	Data Science	A
Std2	AIML	A+
Std3	CS	A++
Std4	Data Science	0
Std5	CSDA	B
Std6	AIML	D

(6, 4)

4

6

name object

course object

```

Grade      object
mark       int64
dtype: object
<class 'pandas.core.frame.DataFrame'>
Index: 6 entries, Std1 to Std6
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0   name    6 non-null      object
1   course  6 non-null      object
2   Grade   6 non-null      object
3   mark    6 non-null      int64
dtypes: int64(1), object(3)
memory usage: 240.0+ bytes
None

```

```

              mark
count  6.000000
mean   79.333333
std    19.106718
min    50.000000
25%    69.000000
50%    81.000000
75%    95.250000
max    99.000000

```

```

      name  course Grade  mark
Std1  Midhun  Data Science    A    98
Std4   Mahi  Data Science    0    99
name      Midhun
course    Data Science
Grade      A
mark       98

```

```

Name: Std1, dtype: object
      name  course Grade  mark
Std1  Midhun  Data Science    A    87
Std2   Madhu      AIML    A+    87
Std3   Akash      CS    A++    67
Std4   Mahi  Data Science    0    99
Std5   Ragu      CSDA    B    50
Std6   Raj      AIML    D    75

```

```

      name  course Grade  mark
Std2  Madhu      AIML    A+    87
Std3  Akash      CS    A++    67
Std4  Mahi  Data Science    0    99
Std5  Ragu      CSDA    B    50
Std6  Raj      AIML    D    75

```

```

      name  course  mark
Std2  Madhu      AIML    87
Std3  Akash      CS    67
Std4  Mahi  Data Science    99
Std5  Ragu      CSDA    50
Std6  Raj      AIML    75

```

```

      name  course  mark
Std5  Ragu      CSDA    50
Std3  Akash      CS    67
Std6  Raj      AIML    75
Std2  Madhu      AIML    87
Std4  Mahi  Data Science    99
      name  course  mark
Std4  Mahi  Data Science    99
Std2  Madhu      AIML    87

```

Std6	Raj	AIML	75
Std3	Akash	CS	67
Std5	Ragu	CSDA	50

99

```
In [37]: import pandas as pd
df = pd.read_csv("people-100.csv", usecols=['Last Name', 'Sex'])
print(df)
print(df.head(5))
print(df.tail(5))
print(df.to_string())
print(df['Sex'])
```

	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male
..	...	...
95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male

[100 rows x 2 columns]

	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male
	Last Name	Sex
95	Barnes	Female
96	Patterson	Female
97	Bray	Male
98	Oconnell	Female
99	Bernard	Male
	Last Name	Sex
0	Terrell	Male
1	Summers	Female
2	Travis	Male
3	Martinez	Male
4	Todd	Male
5	Day	Male
6	Buck	Female
7	Hinton	Male
8	Farrell	Male
9	Downs	Male
10	Ross	Female
11	Newton	Male
12	Blake	Male
13	Nash	Female
14	Farmer	Male
15	Knight	Female
16	Rangel	Male
17	Meadows	Male
18	Paul	Male
19	Mitchell	Female
20	Norton	Female
21	Hatfield	Female
22	Barnes	Male
23	Rhodes	Female
24	Mcneil	Female
25	Levine	Male
26	Payne	Male
27	Vega	Female
28	Buckley	Male
29	Weber	Male
30	Camacho	Female
31	Gallagher	Male
32	Andrews	Female

33	Schmidt	Female
34	Sparks	Female
35	Cook	Male
36	Hurst	Male
37	Rich	Female
38	Wiggins	Female
39	Mckinney	Female
40	Stone	Female
41	Walsh	Female
42	Vang	Female
43	Hill	Female
44	Andrews	Male
45	Pugh	Female
46	Estrada	Female
47	Brady	Male
48	Frey	Male
49	Sweeney	Male
50	Escobar	Female
51	Oconnell	Female
52	Esparza	Female
53	Rivas	Female
54	Payne	Female
55	Rocha	Female
56	Watkins	Female
57	Olsen	Female
58	Nelson	Female
59	Phelps	Male
60	Daugherty	Male
61	Garner	Male
62	Bennett	Male
63	Conway	Female
64	Barnett	Female
65	Harper	Male
66	Ward	Female
67	Rubio	Female
68	Pineda	Male
69	Wu	Male
70	Benton	Male
71	Hull	Male
72	Ritter	Female
73	Briggs	Female
74	Andrews	Female
75	Cunningham	Male
76	Atkinson	Male
77	Reid	Female
78	Stevens	Male
79	Huff	Male
80	Foley	Female
81	Waters	Male
82	Stephenson	Male
83	Reese	Female
84	Williamson	Female
85	Vaughn	Male
86	Dickson	Male
87	Bailey	Male
88	Medina	Female
89	Haney	Female
90	Nixon	Female
91	Hardy	Female
92	Cox	Female


```
93      Cole      Male
94      Guzman   Female
95      Barnes   Female
96      Patterson Female
97      Bray      Male
98      Oconnell  Female
99      Bernard   Male
0      Male
1      Female
2      Male
3      Male
4      Male
...
95      Female
96      Female
97      Male
98      Female
99      Male
Name: Sex, Length: 100, dtype: object
```

In []: